FAST–NUCES Islamabad

Generative Artificial Intelligence

Assignment #1

Question 1: Medical Image Classification
using Deep CNNs

Normal vs. Pneumonia Classification

Group Members: Uzair Majeed (23i-3063)
Faez Ahmed (23i-0598)

Instructor: Dr. Shahela Saif

September 2026

I Introduction

A Problem Statement

- Develop a custom CNN to classify chest X-ray images as Normal or Pneumonia.
- Handle image dimensions, duplicate samples, and class imbalance.
- Compare the custom CNN with pretrained VGG16 and ResNet50 in frozen and fine-tuned settings.

B Motivation

- How CNNs learn image features for chest X-ray classification.
- How preprocessing, augmentation, and hyperparameter affect performance.
- Whether pretrained features has better results compared with a CNN trained from scratch.

C Contributions

- Prepared a cleaned dataset with reproducible 80/10/10 splits and training-based class weights.
- Built a custom CNN and evaluated 30 randomly sampled hyperparameter configurations.
- Studied performance using 25%, 50%, 75%, and 100% of the training data.
- Compared five model configurations using classification metrics and examined ten misclassified images.

II Dataset & Preprocessing

A Dataset Selection and Distribution

To build a custom CNN that classifies chest X-ray images as Normal or Pneumonia. VGG16 and ResNet50 were used only as pretrained comparison models.

We selected the Chest X-Ray Images (Pneumonia) dataset by Paul Mooney.

The downloaded dataset contained 5,856 images in `train`, `val`, and `test` folders. Each folder contained `NORMAL` and `PNEUMONIA` subfolders.

Table 1: Original dataset distribution.

Original split	Normal	Pneumonia	Total
Training	1,341	3,875	5,216
Validation	8	8	16
Test	234	390	624
Total	1,583	4,273	5,856

Initially, 27.03% of images were Normal and 72.97% were Pneumonia. Therefore, the dataset was imbalanced.

label	NORMAL	PNEUMONIA	All
original_split			
test	234	390	624
train	1341	3875	5216
val	8	8	16
All	1583	4273	5856

label	count	percentage
PNEUMONIA	4273	72.97
NORMAL	1583	27.03

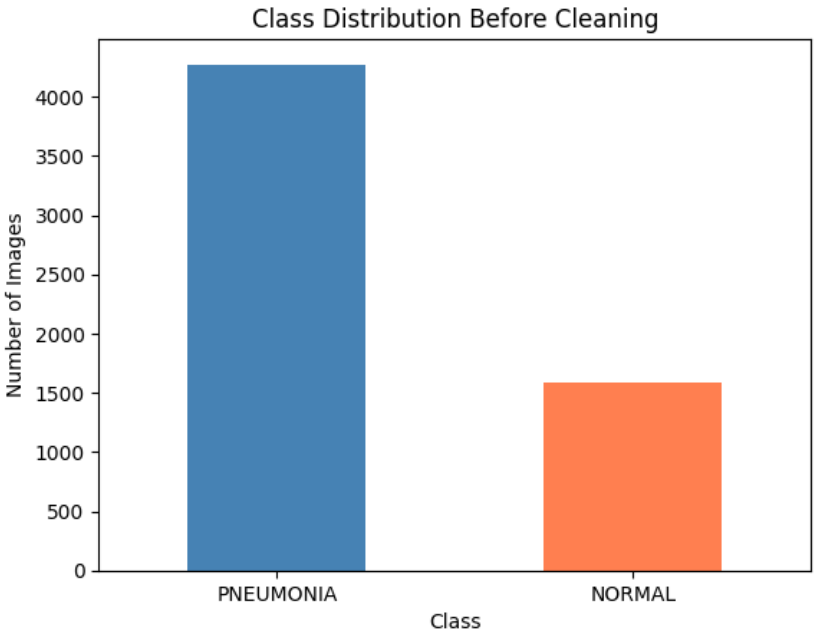


Figure 1: Class distribution before cleaning.

B Preprocessing

Corruption and Duplicate Checks

Since mode=L suggested images were grayscale, We converted images into RGB and generated SHA-256 hashes. Image dimensions comparison also done before resizing to identify duplicates.

- No corrupted images were found.
- 32 duplicate copies were removed.
- No identical images with conflicting labels were found.

```
Original images: 5856
Corrupted images: 0
Extra duplicate copies: 32
Conflicting image groups: 0
Final usable images: 5824
```

Figure 2: Results of corruption, duplicate, and conflicting-label checks.

Dataset After Cleaning

After cleaning, total images were 5,824 : 1,579 Normal and 4,245 Pneumonia. The dataset was still very imbalanced, with 27.11% Normal and 72.89% Pneumonia images.

label	before	after	after_percentage
PNEUMONIA	4273	4245	72.89
NORMAL	1583	1579	27.11

mode	count
L	5541
RGB	283

dtype: int64

	width	height
count	5824.000000	5824.000000
mean	1327.476133	970.225962
std	363.525671	383.299275
min	384.000000	127.000000
25%	1056.000000	688.000000
50%	1280.000000	886.000000
75%	1560.000000	1187.000000
max	2916.000000	2713.000000

Figure 3: Per-class image counts before and after cleaning.

Channel Handling

5541 of the remaining images used grayscale mode L, while 283 used RGB. We converted all images into RGB to use consistent depth of 3 for later comparison with pre-trained models.

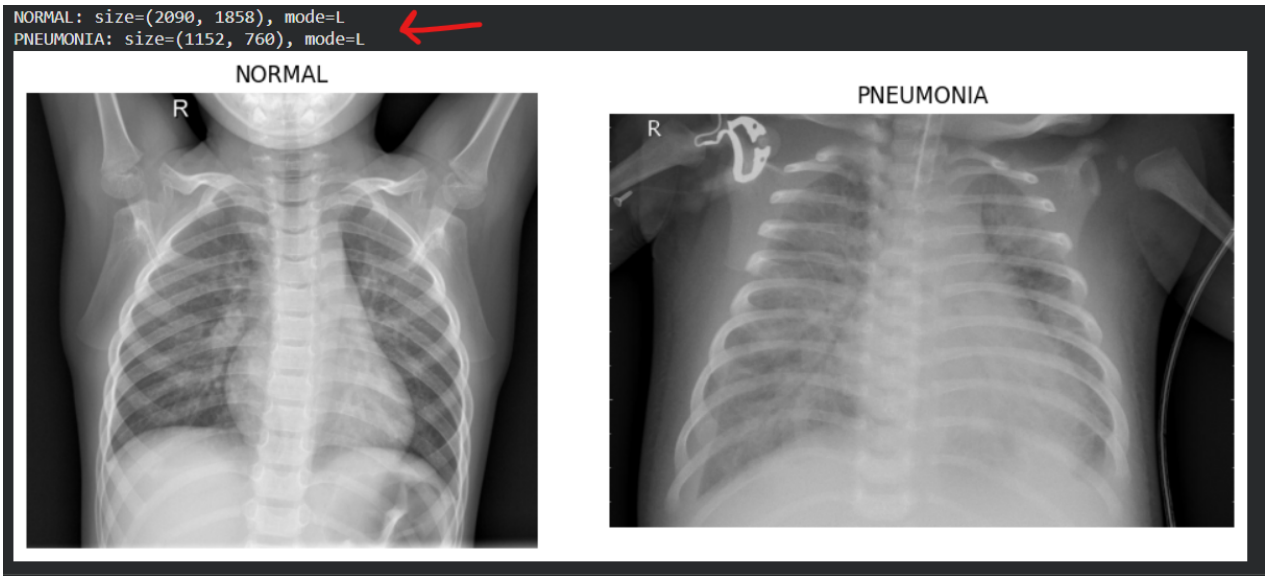


Figure 4: Original channel modes of the retained images.

Image Resizing

Original Images had different dimensions with widths from [384 to 2,916 pixels] and heights from [127 to 2,713 pixels]. So, We resized all of them to 224×224 using bilinear resampling.

```
Image array: (5824, 224, 224, 3)
Label array: (5824,)
Pixel range: 0.0 to 1.0
```

Figure 5: Original image dimensions before resizing.

Normalization

All images were normalized to $[0, 1]$ by dividing pixel values by 255 during preprocessing.

During hyperparameter tuning, we tested two options: retaining the $[0, 1]$ range or applying an additional $2x - 1$ rescaling layer inside the model to obtain $[-1, 1]$.

The configuration with the highest validation AUC used the additional $2x - 1$ rescaling.

Data Splitting and Class Imbalance

After cleaning, we used 80/10/10 split as mentioned per assignment using stratified splitting with seed 42. Stratification preserved the class proportions.

Table 2: Final per-class split counts.

Class	Training	Validation	Test	Total
Normal	1,263	158	158	1,579
Pneumonia	3,396	424	425	4,245
Total	4,659	582	583	5,824

Assertions confirmed that no row indices or exact image hashes were shared between the three splits. These checks establish no exact-image overlap but they do not verify patient-level independence or detect near-duplicates.

Class weights were calculated using training labels only:

$$w_c = \frac{N}{2n_c},$$

where N is the training-set size and n_c is the number of training images in class c.

```
from sklearn.utils.class_weight import compute_class_weight

weights = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0, 1]),
    y=Y_train
)

class_weights = dict(enumerate(weights))
print(class_weights)


# Approximately {0: 1.84, 1: 0.69}
```

```
{0: np.float64(1.844418052256532), 1: np.float64(0.6859540636042403)}
```

```
assert set(train_idx).isdisjoint(val_idx)
assert set(train_idx).isdisjoint(test_idx)
assert set(val_idx).isdisjoint(test_idx)

train_hashes = set(clean_df.iloc[train_idx]["image_hash"])
val_hashes = set(clean_df.iloc[val_idx]["image_hash"])
test_hashes = set(clean_df.iloc[test_idx]["image_hash"])

assert train_hashes.isdisjoint(val_hashes)
assert train_hashes.isdisjoint(test_hashes)
assert val_hashes.isdisjoint(test_hashes)

print("No shared rows or exact duplicate images across splits.")
```

```
No shared rows or exact duplicate images across splits.
```

Figure 6: Class Imbalance and Leakage.

The resulting weights were approximately 1.8444 for Normal and 0.6860 for Pneumonia. This increased the training penalty for mistakes on the minority Normal class.

Data Augmentation

Four techniques were used: rotation, zoom, translation, and brightness adjustment. Two strength policies were considered.

Table 3: Augmentation settings.

Technique	Light	Strong
Rotation	$\pm 7.2^\circ$	$\pm 14.4^\circ$
Zoom factor variation	± 0.10	± 0.15
Horizontal/vertical shift	$\pm 5\%$	$\pm 8\%$
Brightness offset	± 0.10	± 0.15

Brightness offsets refer to the $[0, 1]$ pixel scale. The same zoom factor was used for height and width. Reflected pixels filled exposed image boundaries.

Augmentation was applied only during training. Validation and test images were not augmented. The selected best model used the light policy which will be discussed later on.

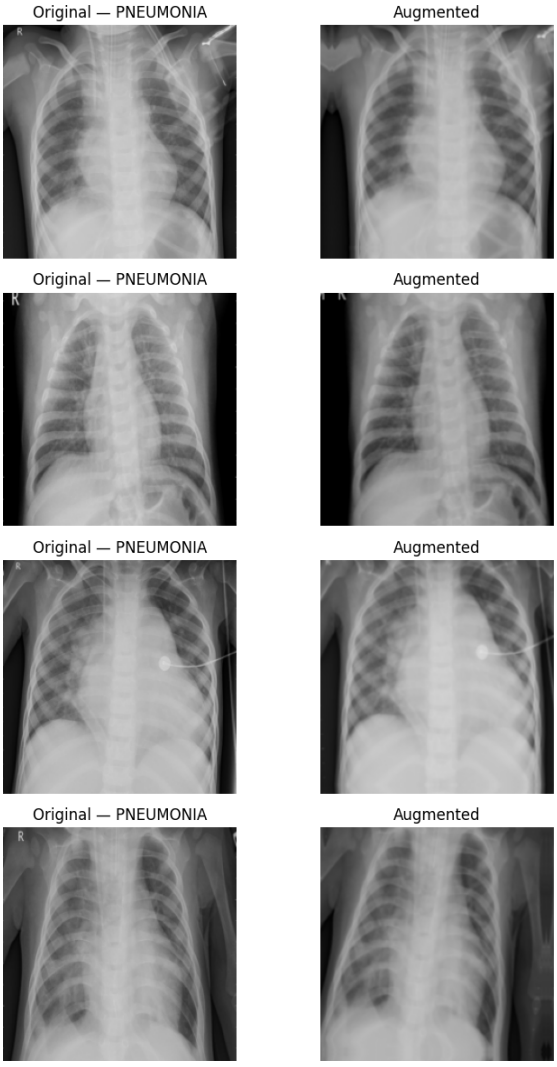


Figure 7: Sample images before and after augmentation.

III Methodology

A Custom CNN Architecture

The primary model was built from scratch using 3 convolution-and-pooling blocks, followed by a dense classifier.

Table 4: Selected custom CNN architecture after grid search.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Augmentation	$224 \times 224 \times 3$	0
Rescaling $(2x - 1)$	$224 \times 224 \times 3$	0
Conv2D, 32 filters, 3×3	$224 \times 224 \times 32$	896
Max pooling, 2×2	$112 \times 112 \times 32$	0
Conv2D, 64 filters, 3×3	$112 \times 112 \times 64$	18,496
Max pooling, 2×2	$56 \times 56 \times 64$	0
Conv2D, 128 filters, 3×3	$56 \times 56 \times 128$	73,856
Max pooling, 2×2	$28 \times 28 \times 128$	0
Flatten	100,352	0
Dense, 64 units	64	6,422,592
Dropout, 0.3	64	0
Dense, sigmoid	1	65
Total model parameters		**6,515,905**
Total optimizer parameters		**13,031,812**
Combined Total parameters		**19,547,717**

- Convolution layers used ReLU activation and same padding.
- Flatten preserved spatially indexed features, but made the dense layer large.
- Dropout and L2 regularization were used to reduce overfitting.
- The sigmoid output used for binary classification.

Completed grid trials: 30 / 30

Model: "sequential"

Layer (type)	Output Shape	Param #
augmentation (Sequential)	(None, 224, 224, 3)	0
rescaling (Rescaling)	(None, 224, 224, 3)	0
conv2d (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 64)	6,422,592
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 19,547,717 (74.57 MB)
Trainable params: 6,515,905 (24.86 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 13,031,812 (49.71 MB)

Figure 8: Model Architecture Summary.

B Pretrained Comparison Models

VGG16 and ResNet50 used ImageNet-pretrained weights. Their original classifiers were replaced by:

$$\text{Global Average Pooling} \rightarrow \text{Dense}(64) \rightarrow \text{Dropout} \rightarrow \text{Sigmoid}.$$

Each architecture was evaluated in two settings:

- **Frozen:** only the new classifier head was trained.
- **Fine-tuned:** training continued from the best frozen checkpoint, with the final convolution block unfrozen.
- **Best Hyperparameter Settings:** The same splits, image size, selected augmentation, class weights, batch size, dropout, L2 strength, epoch cap, and early-stopping rule were used.

Differences:

- Pretrained models used their required ImageNet preprocessing, and fine-tuning used learning rate 10^{-4} instead of 10^{-3}.
- Preprocessing converted $[0, 1]$ RGB inputs back to the $[0, 255]$ scale

Table 5: Executed training epochs for the compared models.

Model	Current stage	Prior frozen stage	Total
Custom CNN	19	0	19
VGG16 frozen	16	0	16
VGG16 fine-tuned	7	16	23
ResNet50 frozen	21	0	21
ResNet50 fine-tuned	12	21	33

IV Experimental Setup

A Training Pipeline and Convergence

- The models used binary cross entropy loss and the Adam optimizer.
- L2 Regularization was applied.
- Training accuracy and ROC-AUC were recorded.
- Checkpoints were saved when validation AUC improved.
- Early stopping monitored the same metric and restored the best weights.
- For the selected custom CNN, the maximum was 30 epochs and patience was 5.

The selected run, `grid_021`, stopped after 19 epochs. Its best checkpoint was at epoch 14, with validation accuracy 0.9570, validation AUC 0.9953, and validation loss 0.1332.

B Hyperparameter Tuning

The defined search space contained $2^8 = 256$ combinations. Because of time and compute limits, 30 combinations were sampled using `ParameterSampler` with seed 42. A fresh CNN was trained for every sampled combination.

This was random search over a discrete parameter grid, not exhaustive grid search. It is a deviation from the assignment's requested grid-search method.

All 30 trials completed. The best configuration was selected using validation AUC.

Table 6: Search options and best values found among 30 trials.

Hyperparameter	Search options	Best value found
Batch size	32, 64	32
Learning rate	10^{-3}, 10^{-4}	10^{-3}
Maximum epochs	15, 30	30
Dropout ratio	0.3, 0.5	0.3
Early-stopping patience	3, 5	5
L2 lambda	10^{-4}, 10^{-3}	10^{-4}
Normalization	$[0, 1]$, $[-1, 1]$	$[-1, 1]$
Augmentation policy	Light, strong	Light

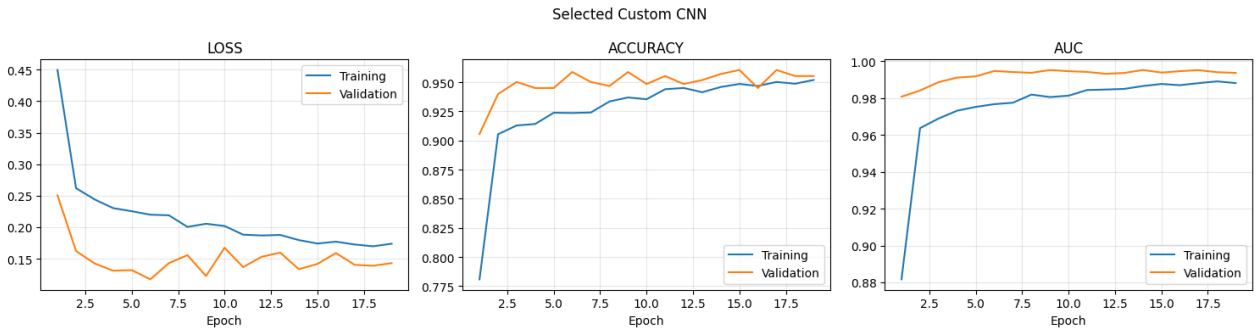


Figure 9: Grid (Random Sampled) Search Curves.

C Effect of Training-Data Volume

Fresh custom CNNs were trained on 25%, 50%, 75%, and 100% of the training split using the selected best hyperparameters. The validation set remained unchanged.

Subsets were nested and approximately class-stratified. Class weights were recalculated for each subset.

Table 7: Training-data volume and observed validation performance.

Training data	Images	Observation
25%	1,164	Performed slightly better than the 50% run.
50%	2,329	Lowest validation accuracy and AUC among the four runs.
75%	3,494	Highest validation accuracy.
100%	4,659	Highest validation AUC; accuracy slightly below the 75% run.

Each subset was trained once, so training variation and checkpoint selection may affect the trend.

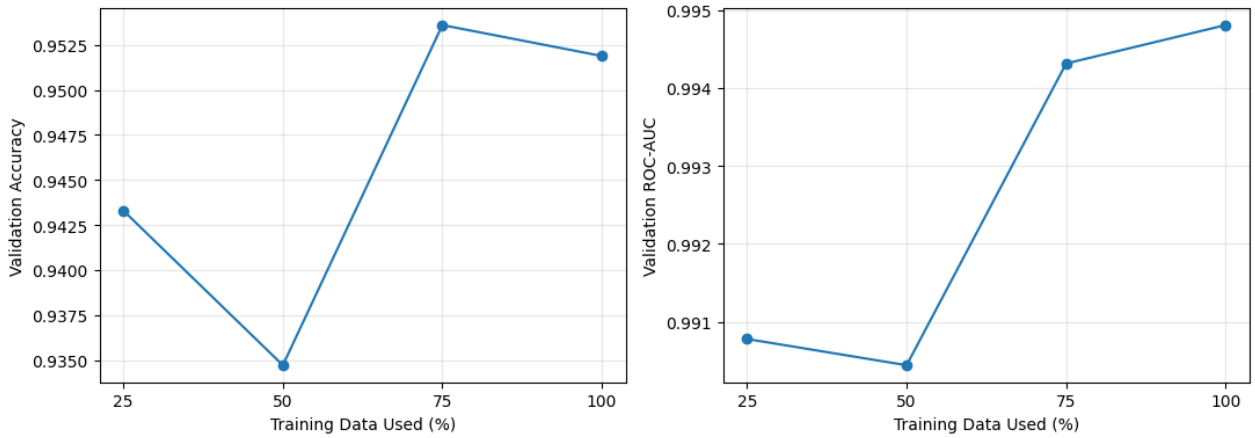


Figure 10: Validation accuracy and ROC-AUC against training-data volume.

V Results

A Training and Validation Curves

- The test set contained 583 images: 158 Normal and 425 Pneumonia.
- A fixed threshold of 0.5 converted sigmoid scores into class predictions.
- Precision, recall, and F1 were calculated for each class.
- Macro averages gave equal weight to both classes.
- ROC-AUC was calculated from prediction scores rather than thresholded labels.

Table 8: Test comparison. Highest values in each metric are highlighted.

Model	Accuracy	Macro P	Macro R	Macro F1	AUC
Custom CNN	0.9537	0.9378	0.9464	0.9420	0.9893
VGG16 frozen	0.9623	0.9539	0.9503	0.9521	0.9888
VGG16 fine-tuned	**0.9691**	**0.9626**	**0.9589**	**0.9608**	0.9939
ResNet50 frozen	0.9571	0.9421	0.9507	0.9463	0.9899
ResNet50 fine-tuned	0.9605	0.9493	0.9511	0.9502	**0.9947**

Table 9: Per-class results. Highest values are highlighted separately for each class, including ties.

Model	Class	Precision	Recall	F1
Custom CNN	Normal	0.9018	0.9304	0.9159
	Pneumonia	0.9738	0.9624	0.9680
VGG16 frozen	Normal	0.9359	0.9241	0.9299
	Pneumonia	0.9719	0.9765	0.9742
VGG16 fine-tuned	Normal	**0.9487**	**0.9367**	**0.9427**
	Pneumonia	**0.9766**	**0.9812**	**0.9789**
ResNet50 frozen	Normal	0.9080	**0.9367**	0.9221
	Pneumonia	0.9762	0.9647	0.9704
ResNet50 fine-tuned	Normal	0.9245	0.9304	0.9274
	Pneumonia	0.9741	0.9718	0.9729

- The custom CNN achieved 95.37% accuracy and 0.9893 AUC.
- Fine-tuned VGG16 had the highest accuracy and macro F1.
- Fine-tuned ResNet50 had the highest AUC.

VGG16 training curves

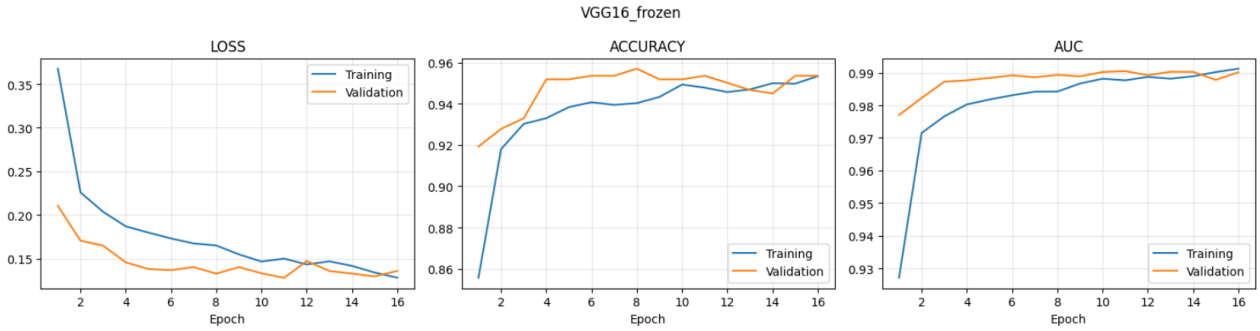


Figure 11: VGG16 frozen training curves.

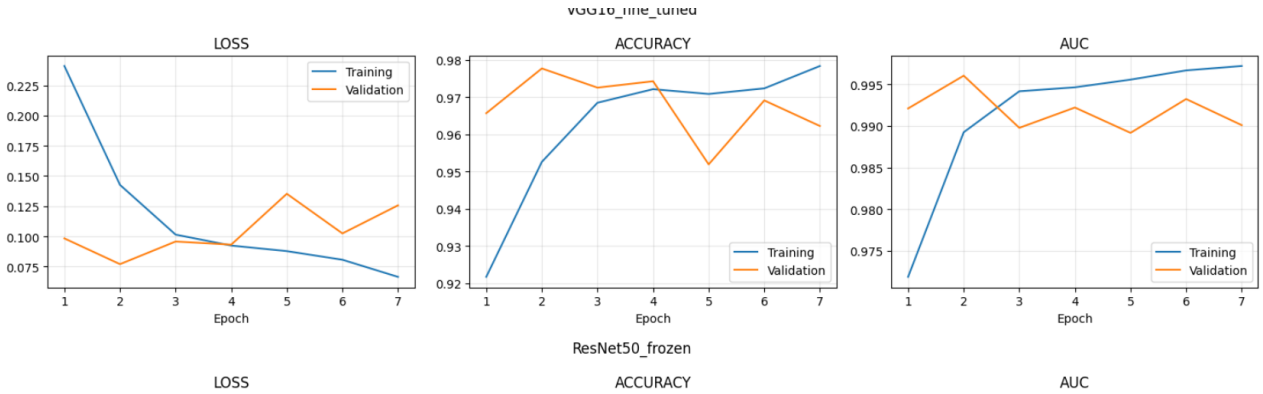


Figure 12: VGG16 fine-tuning curves.

ResNet50 training curves

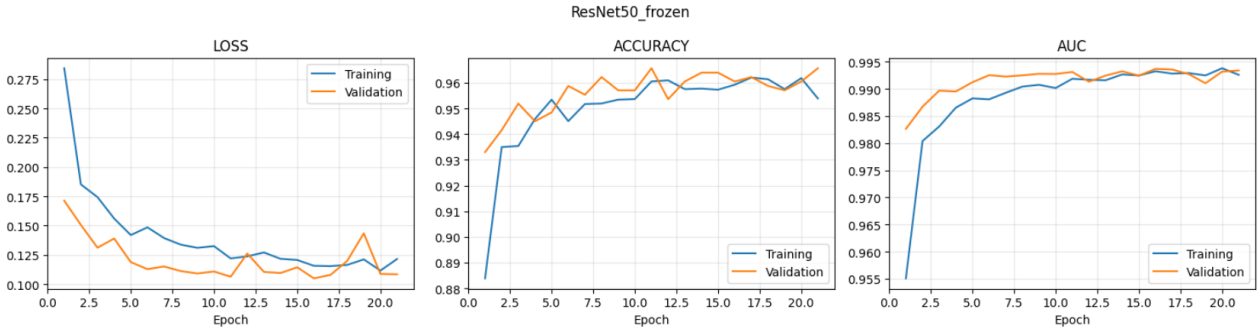


Figure 13: ResNet50 frozen training curves.

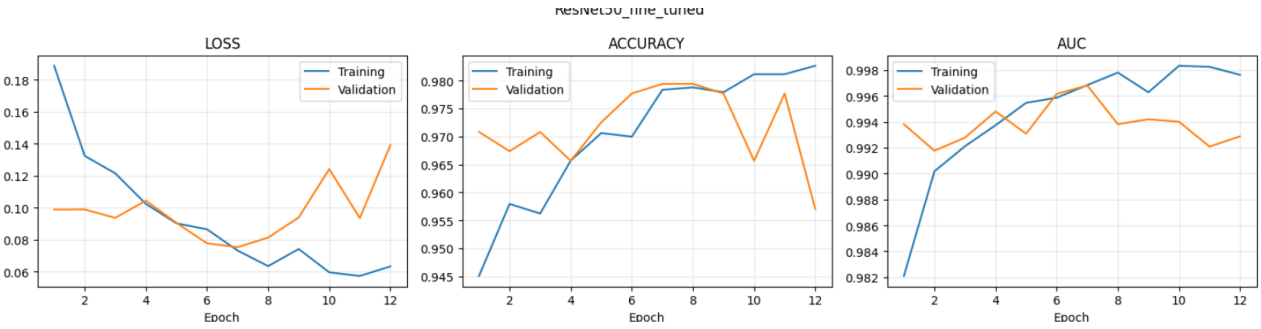


Figure 14: ResNet50 fine-tuning curves.

B Confusion Matrices and ROC Curves

For the custom CNN, the confusion matrix was:

	Predicted Normal	Predicted Pneumonia
Actual Normal	147	11
Actual Pneumonia	16	409

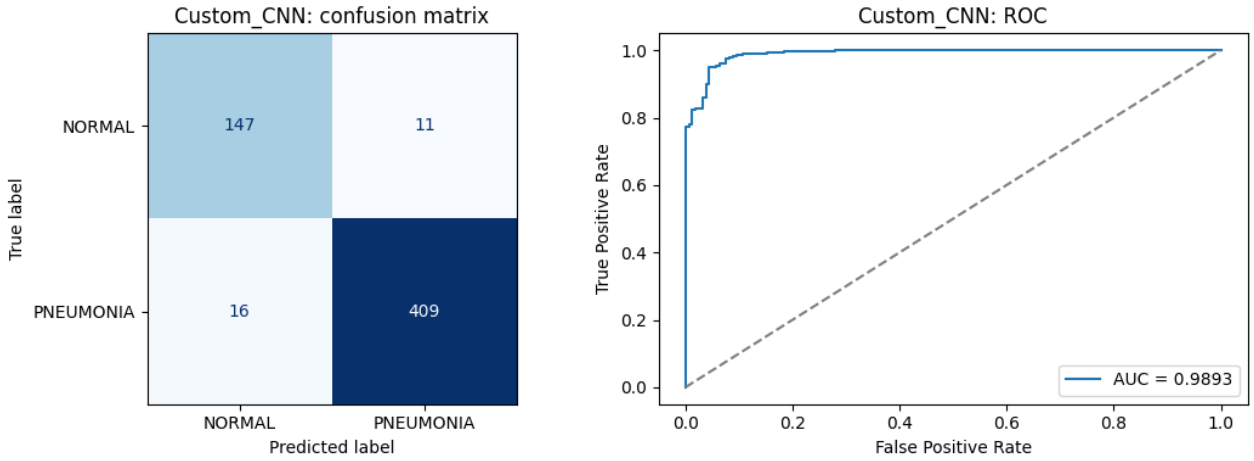


Figure 15: Selected Custom CNN training curves.

VGG16 confusion matrix and ROC curves

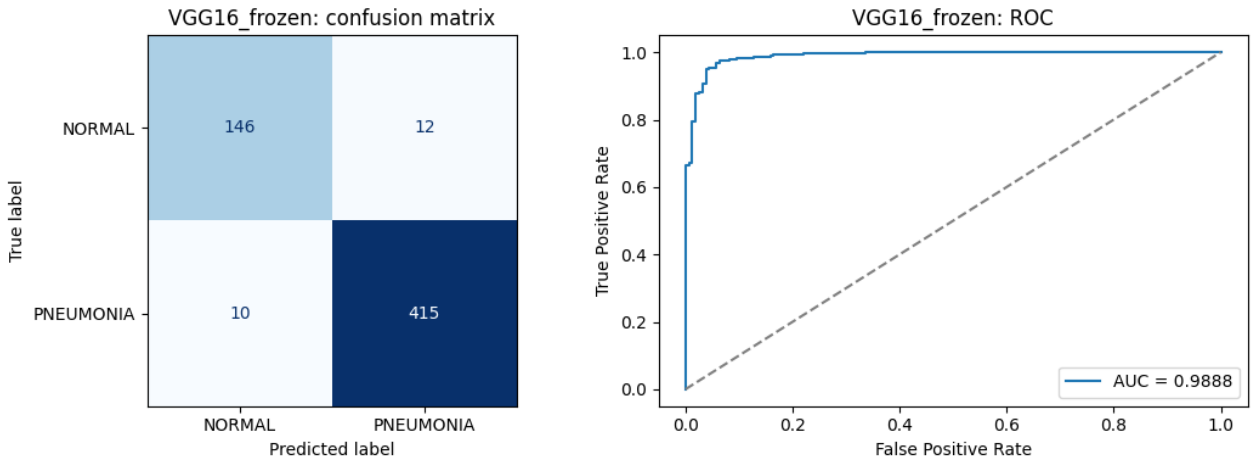


Figure 16: Frozen VGG16 test confusion matrix and ROC curve.

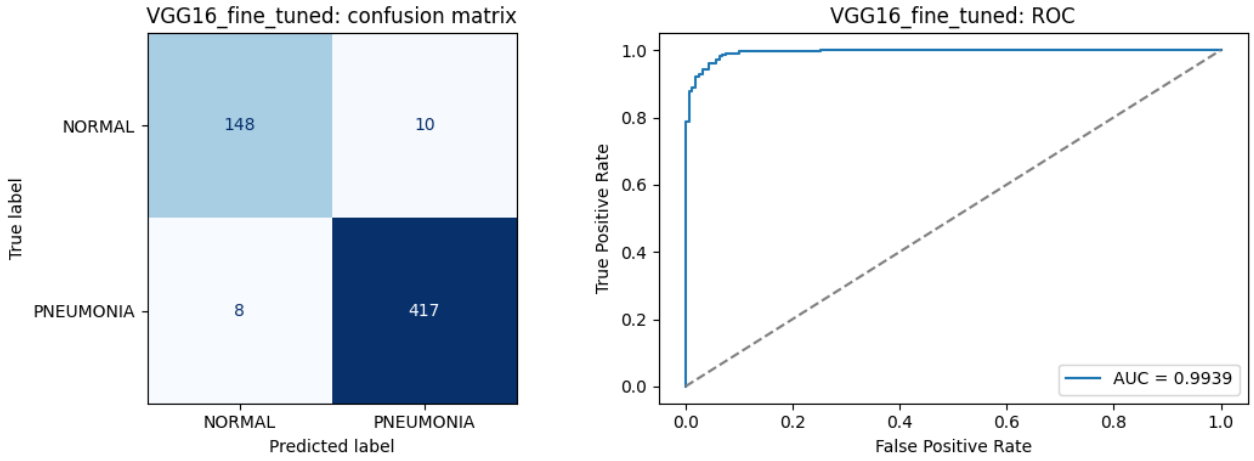


Figure 17: Fine-tuned VGG16 test confusion matrix and ROC curve.

ResNet50 confusion matrix and ROC curves

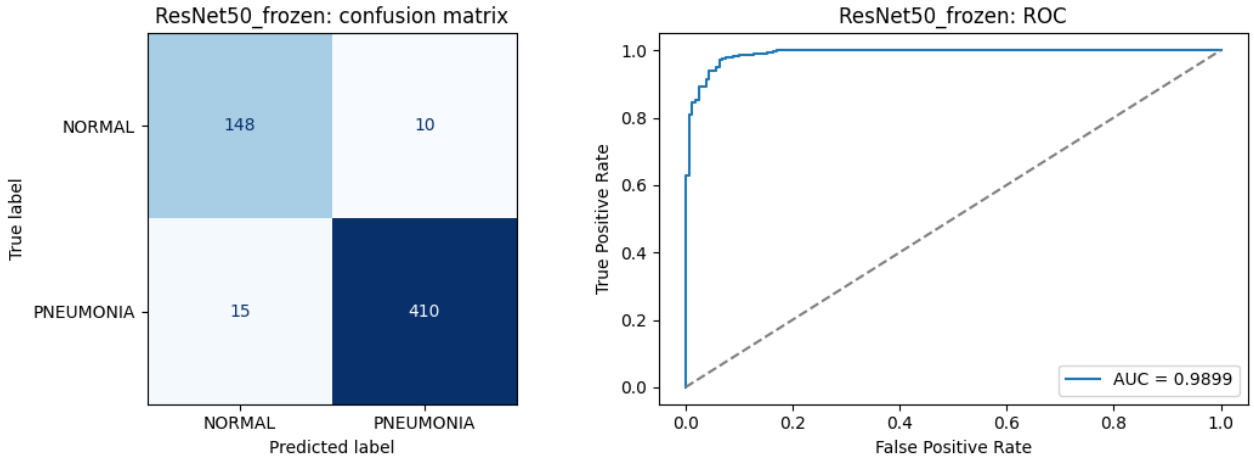


Figure 18: Frozen ResNet50 test confusion matrix and ROC curve.

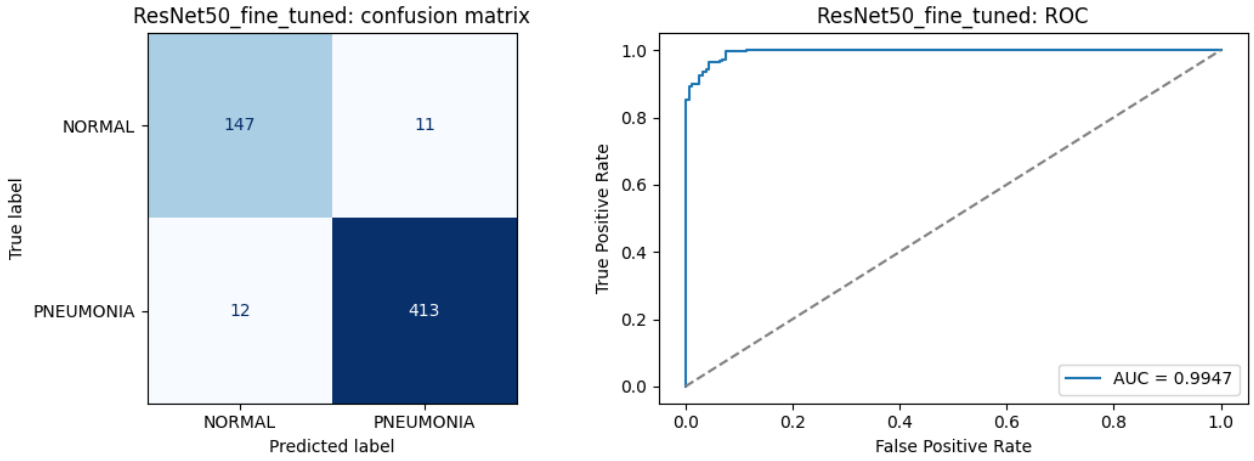


Figure 19: Fine-tuned ResNet50 test confusion matrix and ROC curve.

VI Error Analysis

The custom CNN made 27 test errors:

- The custom CNN made 27 test errors: 11 Normal images were predicted as Pneumonia, and 16 Pneumonia images were predicted as Normal.
- Ten errors were inspected: five from each error type given below.

A Sample Predictions

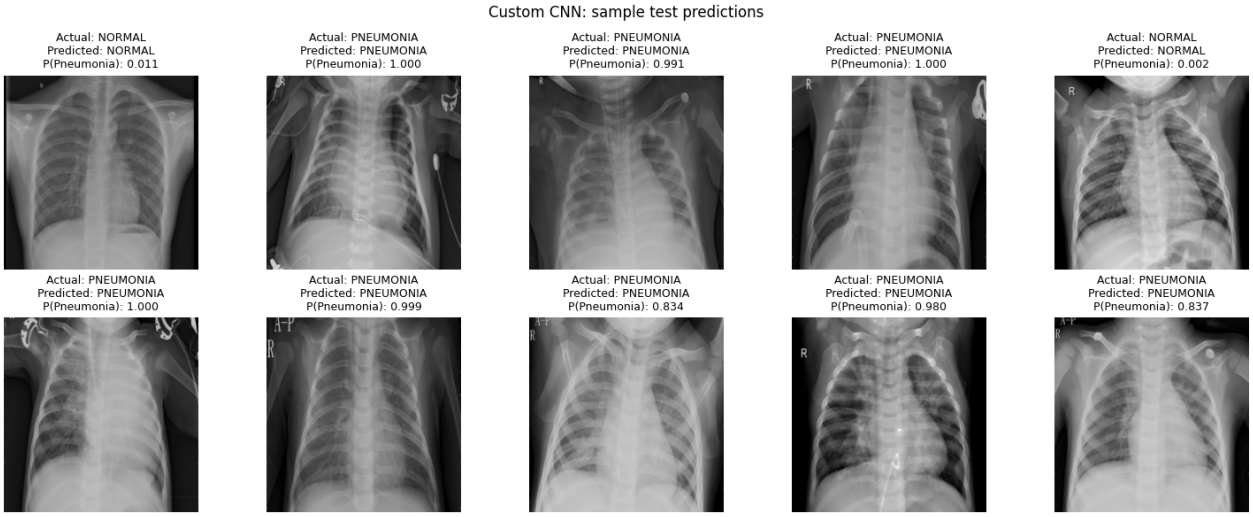


Figure 20: Sample custom CNN predictions with actual labels and Pneumonia scores.

B Missclassified Predictions

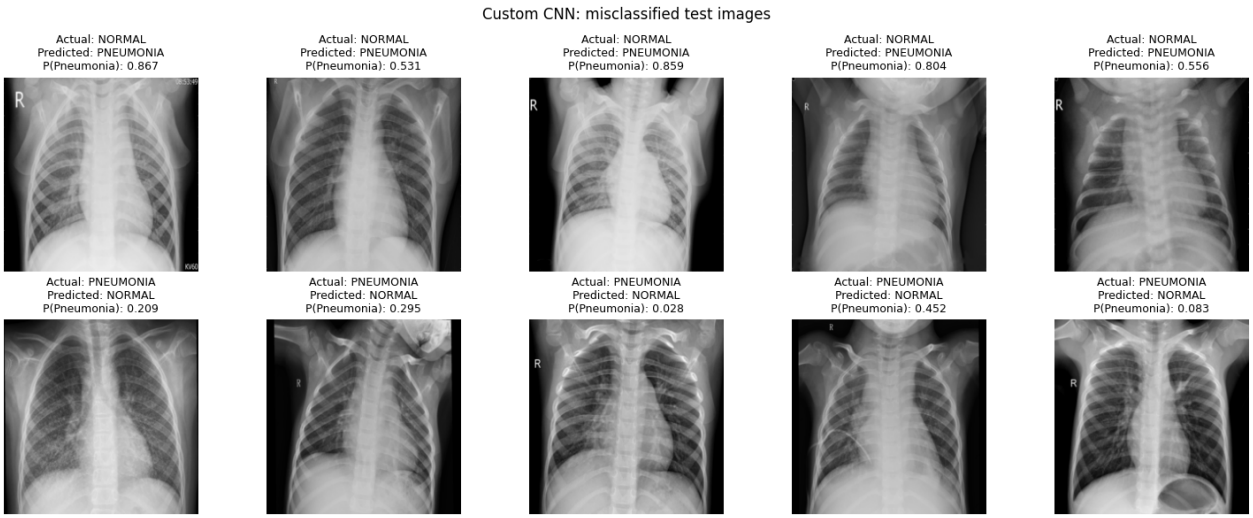


Figure 21: Ten misclassified test images from the custom CNN.

Table 10: Review of ten errors. Observations describe the displayed images, not clinical diagnoses.

ID	Actual	Predicted	Score	Observation
1	Normal	Pneumonia	0.8666	Bright central region and visible side markers. The score is well above the decision threshold.
2	Normal	Pneumonia	0.5312	Dark lung regions with a bright central region. The prediction is close to the threshold.
3	Normal	Pneumonia	0.8590	A bright, relatively low-contrast image with shoulder overlap near the upper chest.
4	Normal	Pneumonia	0.8039	The head and chin overlap the upper image; the chest appearance is asymmetric.
5	Normal	Pneumonia	0.5559	A broad, relatively low-contrast chest view. The score is only slightly above 0.5.
6	Pneumonia	Normal	0.2093	Fine texture is visible across the lung regions, but the model assigns a low Pneumonia score.
7	Pneumonia	Normal	0.2948	Tilted positioning and prominent shoulder structures change the appearance of the chest.
8	Pneumonia	Normal	0.0279	Strong rib contrast is visible. This is the lowest Pneumonia score among the ten selected errors.
9	Pneumonia	Normal	0.4524	A broad bright central region is visible. The prediction is close to the threshold.
10	Pneumonia	Normal	0.0828	Strong rib contrast and asymmetric positioning are visible. The model gives a low Pneumonia score.

C Generalization and Limitations

- The selected custom CNN's validation AUC was 0.9953, compared with test AUC 0.9893.
- Validation accuracy was 95.70%, compared with test accuracy 95.37%.
- The experiment is done on only one dataset. It did not perform on real hospital data.

D Future Work

- Verify patient identities and create patient-independent splits.
- Evaluate on an external chest X-ray dataset.
- Repeat experiments with multiple seeds to measure variation.
- Compare Flatten with global average pooling to reduce parameters.
- Test aspect-ratio-preserving resizing and review its effect on errors.
- Complete a broader search and controlled regularization comparisons.

VII Conclusion

A custom CNN was trained on 5,824 cleaned chest X-ray images. It achieved 95.37% test accuracy and 0.9893 ROC-AUC. Class weighting addressed the imbalance during training, and augmentation introduced variation without changing validation or test images.

Fine-tuned VGG16 achieved the highest test accuracy of 96.91%, while fine-tuned ResNet50 achieved the highest AUC of 0.9947. The custom CNN remained the primary model; pretrained networks served as comparison baselines.